MUSTAFACODE

Lecture 05

Object-Oriented Programming (CS304)

Course Code: **CS304**

Publisher: **mustafacode**

Compiled On: **May 29, 2026**

CS304 OBJECT-ORIENTED PROGRAMMING

Lecture 5:

Lecture No.05

Multiple Inheritance

Inheritance:

Hum ne pichli lecture mein inheritance ke purposes dekhe thay:

• Generalization (general banana)
• Extension ya sub typing (naye features add karna)
• Specialization ya restriction (specific banana ya limit karna)

Abstract aur concrete classes:
• Abstract class = abstract concepts ko represent karti hai
• Concrete class = real / practical concepts ko represent karti hai

Overriding:
Derived classes, base classes ke inherited behavior ko override karti hain.

Overriding ka use hota hai:
• Specialization (kisi cheez ko special banana)
• Extension (features barhana)
• Restriction (limit lagana)
• Performance improvement

05.1 Multiple Inheritance

Kabhi kabhi humein zaroorat hoti hai ke hum ek class mein multiple parent classes ki properties reuse karein.
Is case mein hum ek class ko **ek se zyada classes se inherit** karte hain.

Example 1 – Multiple Inheritance

Ek imaginary makhlooq ka example lein: **Mermaid** (pariyon ki kahaniyon mein use hoti hai).

Mermaid pani (water) mein rehti hai aur is mein dono qualities hoti hain:
• Ek aurat (Woman) ki
• Aur ek machli (Fish) ki

Object Oriented Programming ke perspective se:
Mermaid ko hum do classes se derive kar sakte hain:
• Women class se
• Fish class se

Yani Mermaid = Women + Fish

Is concept ko hi **Multiple Inheritance** kehte hain.

```
class Fish {
};
class Woman {

    };
class Mermaid : public Woman , public Fish {

}
```

Humari **Mermaid class** dono classes yani **Woman** aur **Fish** ki features inherit karti hai.

Misaal ke taur par:

• Woman class mein ek method hota hai **walk()**
• Fish class mein ek method hota hai **swim()**

Ab jab Mermaid class dono classes se inherit karegi, to Mermaid class dono methods use kar sakti hai:

• Mermaid **walk bhi kar sakti hai** (Woman wali property)
• Mermaid **swim bhi kar sakti hai** (Fish wali property)

Yani Multiple Inheritance ki wajah se Mermaid class mein dono behaviors aa jate hain:
walking + swimming

```cpp
#include <iostream>
#include <stdlib.h>
using namespace std;

/*Program to demonstrate simple multiple inheritance*/

class Fish{
public:
 void swim(){
 cout<<"\n In method swim";
 }
};


class Woman{
  public:
    void walk(){
    cout<<"\n In method walk"<<endl;
   }
};


class Mermaid : public Woman,public Fish{

};


int main(int argc, char *argv[]){

     Mermaid mermaid;
/*This Mermaid object will have two implicit objects one of Fish class and one of
Woman class*/
     mermaid.swim();
     mermaid.walk();
     system("PAUSE");
     return 0;
}
```

Advantage of Multiple Inheritance

Jaise simple (single) inheritance mein hota hai, waise hi **multiple inheritance** bhi **redundant code ko kam karti hai**.

Yani:
Hum ek class ko multiple classes se inherit kar ke un ke functions use kar sakte hain,

aur humein un functions ko dobara likhne ki zaroorat nahi hoti.

Disadvantages / Problems of Multiple Inheritance

Lekin is ke **nuksaan zyada hote hain** compared to advantages.

---

1. Increased Complexity (Complexity barh jana)

Amphibious Vehicle ki hierarchy thodi **complex hoti hai**, kyun ke yeh class do classes se derive hoti hai.

Is wajah se:
• Code zyada complex ho jata hai
• Samajhna mushkil ho jata hai

Lekin yeh natural hai, kyun ke amphibious vehicle khud bhi ek **complex cheez** hai.

---

2. Reduced Understanding (Samajh kam ho jana)

Jab class hierarchy complex ho jati hai, to:
• Object model ko samajhna mushkil ho jata hai
• Khas taur par un logon ke liye jo pehli dafa code dekh rahe hon

---

3. Duplicate Features (Duplicate functions ka masla)

Jab hum ek class ko multiple classes se derive karte hain, to:
• Yeh chance hota hai ke dono parent classes mein **same methods** hon

Is se problems paida hoti hain, jaise:
• Confusion hota hai ke kaunsa method use ho
• Ambiguity create hoti hai

👉 Isi ko **duplicate features problem** kehte hain, jo multiple inheritance mein common hoti hai.


Woman
Woman
eat
...
Fish
Fish
eat
...
Mermaid
eat
...
a. Override eat method in Mermaid class
b. Invoke eat operation of desired parent class

```cpp
#include <iostream>
#include <stdlib.h>
using namespace std;

/* Program to demonstrate simple multiple inheritance with duplicate methods */

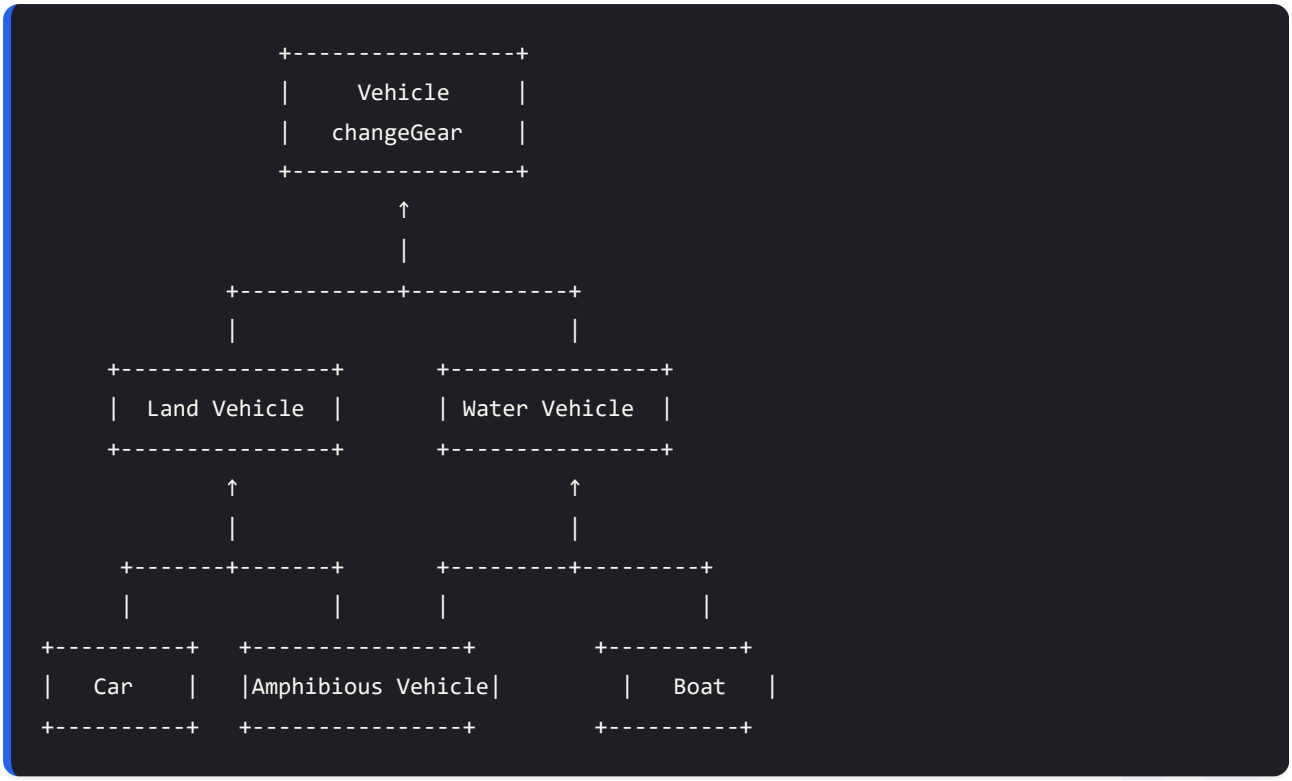
class Fish {
public:
    void eat() {
        cout << "\n In Fish eat method ";
    }
};

class Woman {
public:
    void eat() {
        cout << "\n In Woman eat method \n" << endl;
    }
};

class Mermaid : public Woman, public Fish {
public:
    void eat() {
        cout << "\n In Mermaid eat method " << endl;
        cout << "\n Explicitly calling Woman eat method...." << endl;
        Woman::eat();
    }
};

int main(int argc, char *argv[]) {
    Mermaid mermaid;
    /* This Mermaid object will have two implicit objects: one of Fish class and one of
Woman class */
    mermaid.eat();  // Calling Mermaid eat method
    system("PAUSE");
    return 0;
}
```

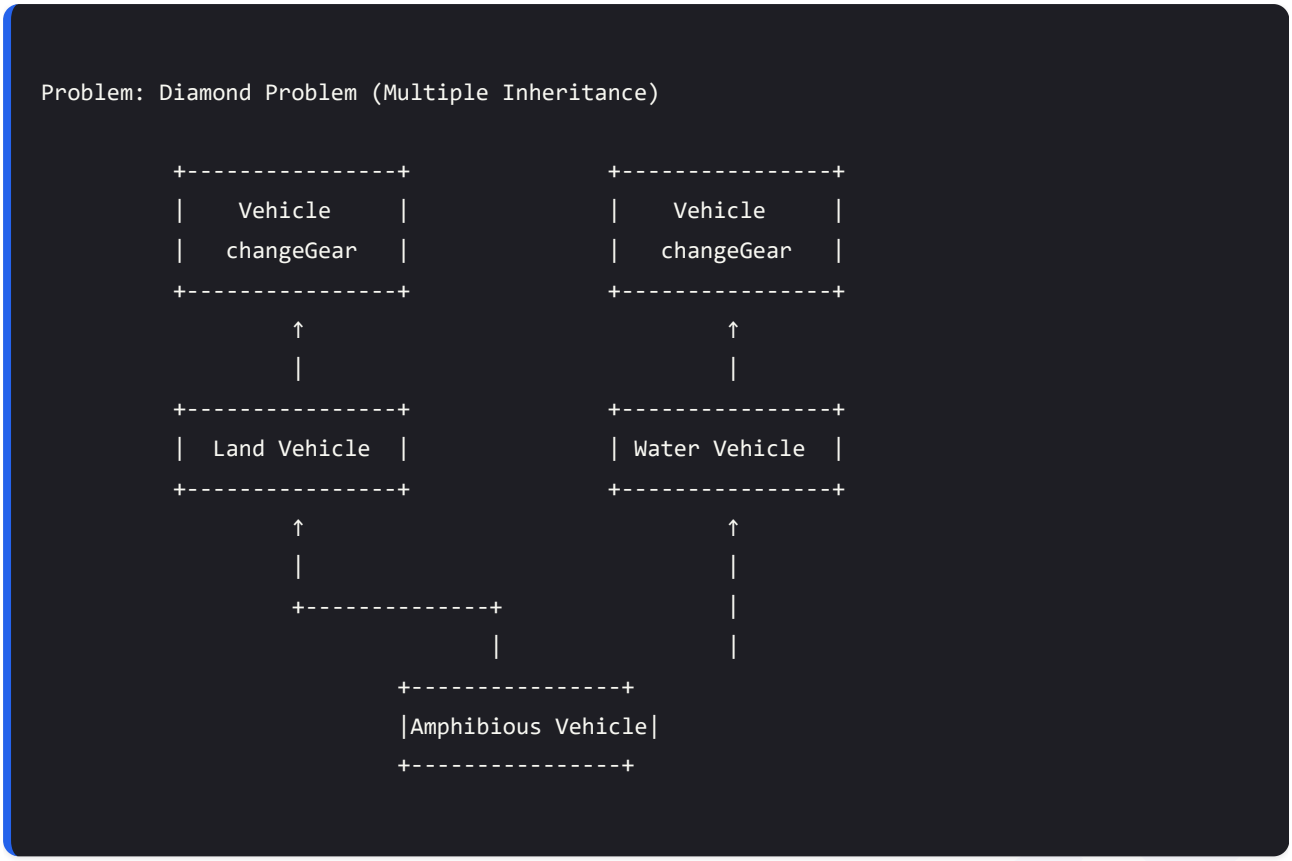
Problem 2: Two instances for same function (Diamond Problem)



Yahan **Amphibious Vehicle** ke paas `changeGear` function ki **do copies** hongi.

Aisa is liye hoga kyun ke is ke paas **Vehicle class ke do objects** honge:
• Ek **Land Vehicle** ke through
• Ek **Water Vehicle** ke through

Yani Amphibious Vehicle mein Vehicle class do dafa inherit ho rahi hai,
is wajah se `changeGear()` function bhi **duplicate ho jata hai**.



Actual Memory Layout

Compiler yeh decide nahi kar paata ke Amphibious Vehicle ko kaunsa changeGear() function inherit karna chahiye.

Is liye woh error generate kar deta hai, kyun ke same method ki do copies hoti hain.

Error kuch is tarah hota hai:

error: request for member `changeGear' is ambiguous

error: candidates are: void Vehicle::changeGear()
void Vehicle::changeGear()

Execution terminated

eska solution ya hai ham class ka name sath ma bata da direct us class ka name sa accesss kar la

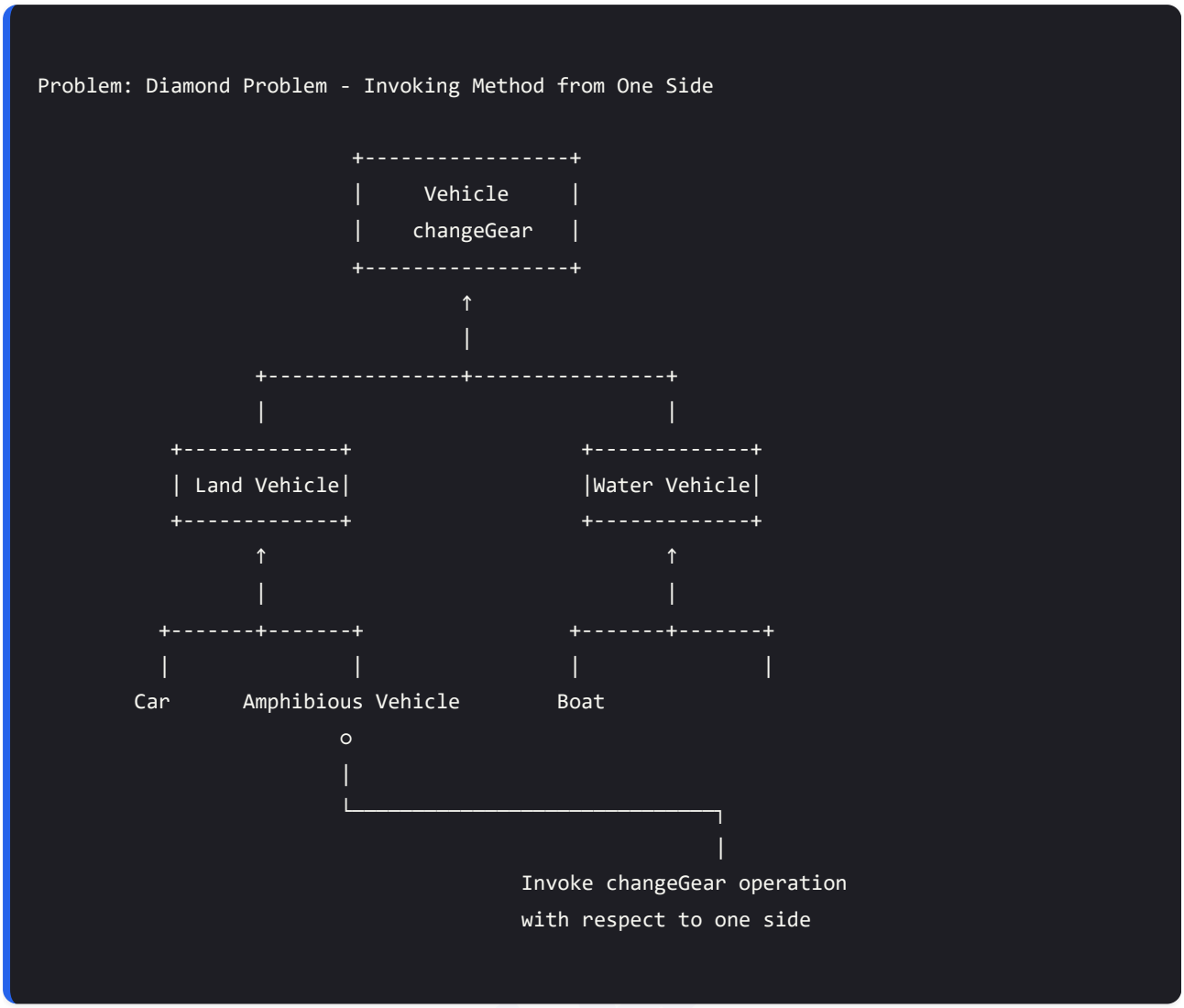
Solution to Diamond Problem

Kuch programming languages diamond hierarchy ko allow hi nahi karti.

Aur kuch languages aisa mechanism deti hain ke: • Hum ek side ki characteristics ko ignore kar sakte hain

Diamond Problem solve karne ke 2 tareeqay hote hain:

• Virtual Inheritance • Non-Virtual Inheritance



Association

Different objects ka aapas mein interaction (ya problem domain mein relation) ko **association** kehte hain.

Object Oriented Model mein, objects ek dusre ke sath interact karte hain taake koi useful kaam perform kar saken. Jab hum in objects (entities) ko model karte hain, to unke darmiyan relation ko **association** ke zariye show kiya jata hai.

Aksar ek object dusre kai objects ko services provide karta hai. Ek object dusre objects ke sath association rakhta hai taake wo apne tasks unhein delegate (sop) kar sake.

Is association ko diagram mein ek line ke through represent kiya jata hai, jo arrow head ke sath ya bina arrow head ke bhi ho sakti hai.

05.2 Association ki Kismein (Kinds of Association)

Association ki do main types hoti hain, jinhein aagay aur bhi divide kiya jata hai:

1. Class Association
2. Object Association

---

1. Class Association

Class association ko **Inheritance** ke zariye implement kiya jata hai. Inheritance objects ke darmiyan **generalization / specialization** relationship ko implement karta hai. Is liye inheritance ko class association bhi kaha jata hai.

• Public inheritance ki case mein yeh **"IS-A" relationship** hota hai.
• Private inheritance ki case mein yeh **"Implemented in terms of" relationship** hota hai.

Yeh relationship ensure karta hai ke base class ke public members derived class ko available hon (public inheritance ki case mein).

Jab hum inheritance wali classes ke objects banate hain, to hum indirectly inheritance relationship ko objects ke level par bhi apply kar rahe hote hain. Is case mein derived class ke objects ke andar base class ke attributes aur methods bhi include ho jate hain.

---

2. Object Association

Object association ka matlab hai ke ek class ke independent (standalone) objects ka dusri class ke objects ke sath interaction.

Is ki teen types hoti hain:

• Simple Association
• Composition
• Aggregation

05.3 Simple Association

Do interacting objects ke darmiyan koi intrinsic (asal ya permanent) relationship nahi hota. Yeh objects ke darmiyan **sab se weak link** hota hai. Yeh ek reference hota hai jis ke zariye ek object dusre object se interact karta hai.

Misal ke taur par:
Customer cashier se cash leta hai
Employee company ke liye kaam karta hai
Ali ek ghar mein rehta hai
Ali ek car chalata hai

Aam tor par isay “simple association” ke bajaye sirf “association” bhi kaha jata hai.

Simple Association ki Kismein

Simple association ko do tareeqon se divide kiya jata hai:

• Navigation (direction) ke hisaab se
• Number of objects (cardinality) ke hisaab se

Navigation ke hisaab se Simple Association ki Kismein

Navigation ke hisaab se association ki do types hoti hain:

a. One-way Association
b. Two-way Association

---

a. One-way Association

One-way association mein hum sirf ek direction mein hi navigate kar sakte hain. Isay arrow ke zariye represent kiya jata hai jo server object ki taraf point karta hai.

Examples:

```
1. Association Example - "lives-in"

          +------+          1          1          +------+
          | Ali  | ----------------------> | House |
          +------+     lives-in             +------+

   • Ali lives in a House


2. Association Example - "drives"

          +------+          1          1          +------+
          | Ali  | ----------------------> |  Car  |
          +------+      drives              +------+

   • Ali drives his Car
```

b. Two-way Association

Two-way association mein hum dono directions mein navigation kar sakte hain. Isay diagram mein ek simple line ke zariye represent kiya jata hai jo associated objects ke darmiyan hoti hai.

Examples: • Employee company ke liye kaam karta hai
• Company employees ko employ karti hai

Two-way Association ka example: • Yasir Ali ka friend hai
• Ali Yasir ka friend hai

Simple Association ki Kismein (Cardinality ke hisaab se)

Cardinality ke hisaab se association ki following types hoti hain:

a. Binary Association
b. Ternary Association
c. N-ary Association

---

a. Binary Association

Binary association mein do objects ke darmiyan relation hota hai.

Examples: • Employee works-for Company

- 1

• Ali drives Car
1 *

• Ali lives-in House
1 1

• Yasir friend Ali
1 1

Binary Association

Yeh association **exactly do classes** ke objects ko relate karti hai. Isay ek line ya arrow ke zariye represent kiya jata hai jo associated objects ke darmiyan hota hai.

Example:
• “works-for” association sirf do classes ko associate karti hai
• “drives” association bhi sirf do classes ko associate karti hai

---

b. Ternary Association

Yeh association **exactly teen classes** ke objects ko associate karti hai. Isay diagram mein **diamond shape** ke sath lines jor kar represent kiya jata hai jo associated objects tak jati hain.

Example:
• Teen different classes ke objects aapas mein related hote hain

---

c. N-ary Association

Yeh association 3 ya us se zyada classes ke darmiyan hoti hai. Is ki practical examples bohat rare hoti hain.

05.4 Composition

Ek object dusre chotay objects (smaller objects) se mil kar bana hota hai. “Part” objects aur “Whole” object ke darmiyan jo relationship hota hai usay **Composition** kehte hain.

Composition ko diagram mein **filled diamond (bhara hua diamond)** ke sath represent kiya jata hai, jo composer (whole) object ki taraf hota hai.

Example – Composition of Ali

• Ali → Car
• Ali car drive karta hai

```
## **Example Relations:**

• Employee → Company (works-for)
• 1 → *
---


### Simple & Clean Text Version:

```markdown
Composition Example 1: Human Body

          Head (1)
             ↓
   Arm (2) — Ali — Leg (2)
             ↓
          Body (1)


Composition Example 2: Chair

          Back (1)
             ↓
          Chair
      /     |     \
   Arm(2)  Seat(1)  Leg(4)
```

Composition is a stronger relationship:

- Composition is a stronger relationship, because
- Composed object becomes a part of the composer
- Composed object can't exist independently

Example I

Ali mukhtalif body parts se mil kar bana hota hai.
Yeh body parts Ali ke baghair independent taur par exist nahi kar sakte.

Example II

Chair ka structure mukhtalif parts se mil kar bana hota hai.
Yeh parts bhi independently exist nahi kar sakte.

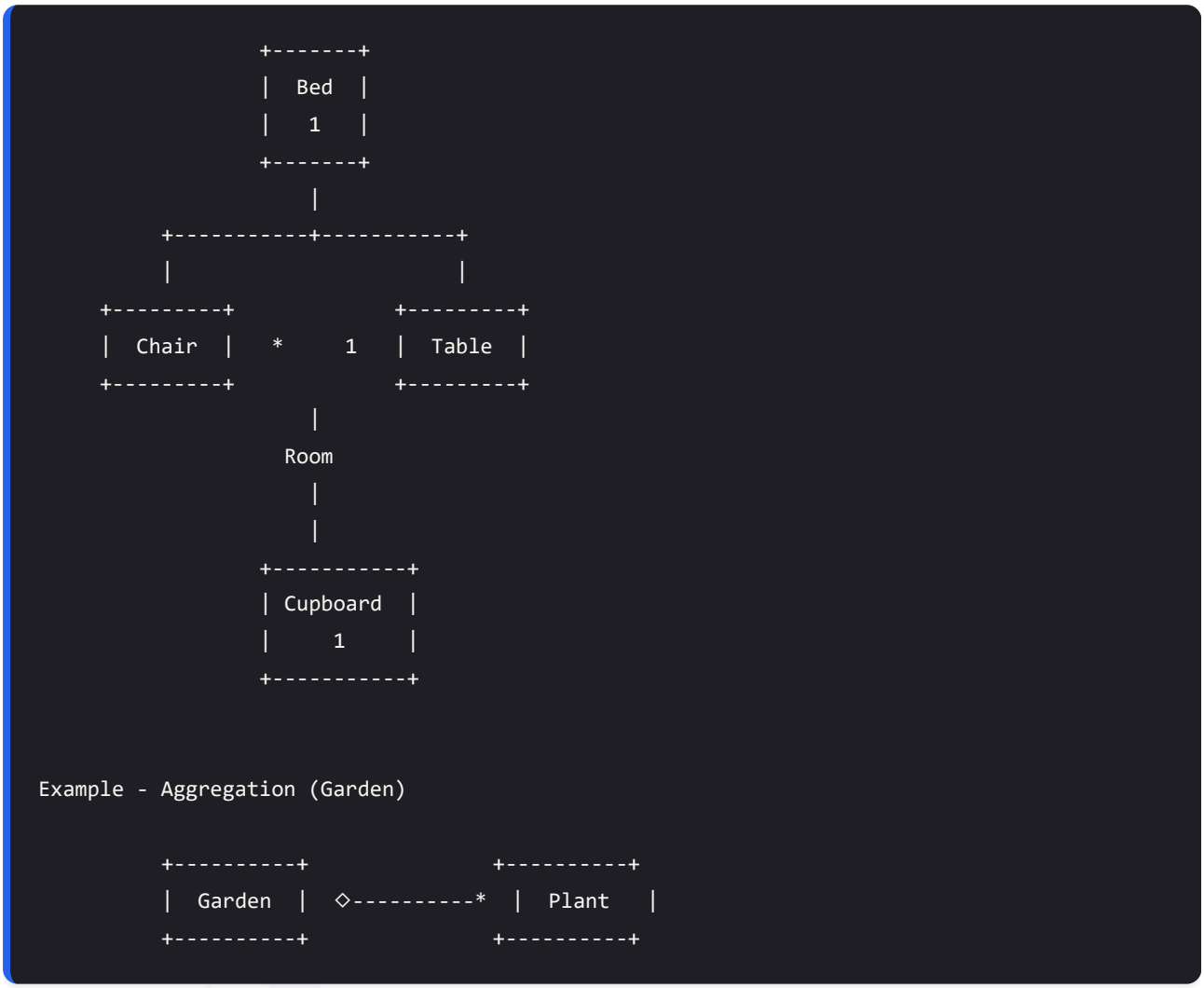
```

05.5 Aggregation

Ek object apne andar doosray objects ka ek collection (aggregate) rakh sakta hai. Container object aur contained objects ke darmiyan jo relationship hota hai usay **aggregation** kehte hain.

Aggregation ko diagram mein ek line ke sath **unfilled diamond (khali diamond)** ke zariye represent kiya jata hai, jo container object ki taraf hota hai.

Example - Aggregation (Room)



Aggregation ek weaker relationship hai

Aggregation ek **kamzor (weaker) relationship** hoti hai, kyun ke:

• Aggregate object container ka asal (intrinsic) hissa nahi hota
• Aggregate object independently bhi exist kar sakta hai

Example I

Furniture room ka intrinsic hissa nahi hota.
Furniture ko ek room se doosray room mein shift kiya ja sakta hai, is liye yeh kisi specific room ke baghair bhi independent taur par exist kar sakta hai.

Example II

Plant garden ka intrinsic hissa nahi hota.
Isay kisi aur garden mein bhi lagaya ja sakta hai, is liye yeh kisi specific garden ke baghair bhi independent taur par exist kar sakta hai.